

QA1

我稍微懂一点，“摊销费用不能随便设，必须符合你想证明的复杂度的量级”再讲解一下表扩张，用记账方法和势能方法分析，我感觉还是没有理解。

好的，我们现在专门针对 **动态表扩张 (Dynamic Array / Vector 扩容)**，用 **记账方法** 和 **势能方法** 做一次 **循序渐进、非常具体、从直觉到数学的完整分析**。

并且我会帮助你理解：

“为什么摊销费用不能随便设？为什么要设成这个值？为什么这个值就能证明 $O(1)$ ？”

这次会比前面的讲解更深一步，让你真正“懂”摊销分析。

🌟 动态数组问题（统一背景）

数据结构：动态数组 (C++ vector、Java ArrayList、Python 内部列表)

操作：push_back(x)，加入新元素

真实代价 c_i ：

- **普通插入**：只需写数组位置 $\rightarrow c_i = 1$
- **扩容插入**：需要
 - i. 分配新数组 (2 倍大小)
 - ii. **复制 m 个旧元素**
 - iii. 写入新元素 $\rightarrow c_i = m + 1$

我们需要证明：

push_back 的摊销时间是 $O(1)$ 。

Part 1: 记账方法 (Accounting Method)

记账方法的思想:

“每次操作我收一个固定的摊销费用，用多余的钱存起来，以便将来扩容时用。”

你想证明 $O(1)$ 摊销时间 $\rightarrow$ 意味着你必须选择一个 **常数 k** 作为每次操作的摊销费用。

整个任务:

找到一个最小的常数 k ，使得账户余额永远不会为负。

✓ 第一步：每次普通插入后，需要为未来扩容“存钱”吗？

想象扩容发生:

假设当前容量为 m ，要扩容到 $2m$ 。

导致复制的代价为 m 。

这些复制工作，是前面 m 次插入积累出来的，所以这 m 次必须给“复制费用”。

每个元素被复制 1 次，因此每个元素插入时至少要贡献 1 元。

此外，每次普通插入的真实代价为 1。

所以每次插入至少需要:

- 1 (当前插入)
- 1 (未来复制该元素)

$\rightarrow$ 最小摊销费用必须 ≥ 2 。

但这样够吗？

✓ 第二步：测试 $k=2$ 是否可行

假设每次插入我收费 2:

- 1 元用于当前插入 (写入元素)
- 1 元存起来，用于将来复制

到了扩容那一刻：

- 我需要复制 m 个元素 $\rightarrow$ 需要 m 元
- 我之前存了 m 元（每个元素都存 1）

完全够用。

但还有一步：

扩容插入还需要执行一次“当前插入”的真实代价 1

这 1 元怎么办？

刚好那次操作也收取 2 元，其中 1 元用于当前插入，另外 1 元存起来（留给未来下一次扩容）。

✓ 第三步：再检查一次收支（关键）

容量为 m 、有 m 个元素时，需要扩容：

- 已存金额： m 元
- 复制代价： m
- 插入代价：1

本次收费：2

- 1 用于当前插入
- 复制使用之前存的钱
- 余额变： $m - m + (2 - 1) = 1$

余额不负。

✓ 结论： $k = 2$ 不够（还差 1）

实际上你会发现：

当扩容之后新容量变成 $2m$ ，元素变成 $m+1$ ，此时我们希望余额仍然保持 $\geq$ 元素数量（为什么？为了以后每个元素复制时还有 1 元）。

所以 $k=2$ 不足，你会得到余额 $<$ 元素数。

最终经过严格计算（我等下用势能法推导给你看），你会得到：

$k = 3$ 是最小可行的摊销费用。

✓ 最终记账方法结论

每次操作收费 3：

- 1 元支付当前插入
- 2 元存起来，将来用于扩容时的复制

扩容时：

- 已存储的钱足够覆盖复制
- 新操作的 3 元足以覆盖插入
- 账户余额永不为负

因此：

每次操作摊销代价 = 3 = $O(1)$

🌟 Part 2：势能方法（Potential Method）

势能方法是记账方法的“数学版”。

✓ 定义势能函数（关键）

我们选：

$$\Phi = 2 \cdot \text{size} - \text{capacity}$$

理由：

- 当数组快满 $\rightarrow \text{size} \approx \text{capacity} \rightarrow \Phi$ 大 $\rightarrow$ 状态“紧张” $\rightarrow$ 表示要扩容 $\rightarrow$ 应该提前“存”势能
- 扩容后 capacity 变大 $\rightarrow \Phi$ 变小 $\rightarrow$ 表示“释放势能”用于支付复制成本

这和记账方法里“存钱—花钱”的思想完全一致。

✓ 分两种情况计算摊销代价

摊销代价定义：

$$\hat{c}_i = c_i + (\Phi_i - \Phi_{i-1})$$

Case 1: 普通插入（不扩容）

真实代价：

$$c_i = 1$$

势能变化：

- size +1 $\rightarrow \Phi$ 增加 2
- capacity 不变

所以：

$$\hat{c}_i = 1 + 2 = 3$$

Case 2: 扩容插入

真实代价：

$$c_i = m + 1$$

扩容前：

$$\Phi_{old} = 2m - m = m$$

扩容后：

$$\Phi_{new} = 2(m + 1) - 2m = 2$$

势能变化：

$$\Phi_{new} - \Phi_{old} = 2 - m$$

摊销代价：

$$\hat{c}_i = (m + 1) + (2 - m) = 3.$$

☀ 势能法的结论

无论普通插入还是扩容插入：

$$\hat{c}_i = 3 = O(1)$$

势能方法也证明：

push_back 的摊销复杂度是 $O(1)$

☀ Part 3：你真正要理解的关键点

① 为什么 k 不能随便设？

因为：

- 若设得太大：摊销代价就变成 $O(k)$ ，没有意义
- 若设得太小：余额会变负，不合法

要证明 $O(1)$ ，你必须找到恰当的常数 k。

② 记账方法本质是算“每次操作要留下多少钱用于未来的昂贵步骤”

在动态扩容中：

- 普通插入必须留下“未来复制这个元素所需要的钱”
- 因为扩容时每个元素将被复制一次

因此每个元素必须存至少 1 元。

写当前插入本身也要 1 元。

再加上扩容后保持势能平衡的需要 → $k=3$ 是最小可行的数。

③ 记账法与势能法完全一致，只是写法不同

记账方法：

- 手工分析“需要多少存款才能支付未来的昂贵操作”

势能方法：

- 用 Φ 函数自动编码“当前状态距离昂贵操作的距离”

它们给出的摊销代价完全一致（都是 3）。

QA2

你的分析我看懂了，尤其是记账分析。但是势能方法没有完全理解，万一算出来的插入一个元素和扩容插入的摊销代价 $\hat{c}_i$ 不一样呢？那么这时候摊销复杂度又是什么？还有，势能函数为什么是这个？如果遇到其他问题，我如何在短时间内确定势能函数

下面我会从你刚才的困惑入手，深入解释你最关心的三件事：

1. 势能方法里，普通插入和扩容插入得到的摊销代价 ($\hat{c}_i$) 如果不同怎么办？最终摊销复杂度怎么算？
2. 势能函数为什么要设为现在这个样子？
3. 如果遇到新的问题，我该如何快速设计势能函数？（给你一套可操作的方法论）

1. 若普通插入和扩容插入得到的摊销代价不同怎么办？

势能方法中，每个操作的摊销代价是：

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$$

显然，一般会有：

- 普通插入： $\hat{c}_i = \text{常数}$ （比如 3）
- 扩容插入： $\hat{c}_i = \text{更大的常数值}$ （比如 5）

这完全正常，而且就是势能方法的本意！

那么最终摊销复杂度是什么？

摊销复杂度考察的是**所有操作的平均代价上界**。

即使两种 $\hat{c}_i$ 不同，只要**每个 $\hat{c}_i$ 都是 $O(1)$** ，我们就认为摊销时间是 $O(1)$ 。

换句话说，摊销分析并不是要求 *每个操作的摊销代价相同*，而是要求 **所有摊销代价都有常数上界**。

例如对于表扩张你算出来可能是：

操作类型	实际成本 c_i	摊销成本 $\hat{c}_i$
普通插入	1	3
扩容插入	k+1（复制大量元素）	5

虽然 3 和 5 不一样，但它们都是 **常数**。

因此摊销复杂度 = 最大的摊销代价 = $O(1)$ 。

不需要只出现单一数字，比如“都是 3”，只要都被某个常数界住就可。

2. 为什么势能函数要选为 $\Phi = 2n - m$ （或类似形式）？

这里给你一个直觉解释：

• 你的势能必须做到两件事：

1. 普通插入后，势能增长要足够大，以抵消未来扩容的开销的部分
2. 扩容完成后，势能要下降，释放之前积累的“能量”来支付复制的高额代价。

• 选 $\Phi = 2n - m$ 有什么意义？

下面解释其中每一项：

- n = 当前使用元素数
- m = 当前表容量

扩容时，数组容量通常翻倍，因此扩容后：

- m 会变得很大
- 刚扩容完时通常有 $n \approx m/2$

所以：

✚ 扩容后：

$$\Phi = 2n - m \approx 2(m/2) - m = 0$$

势能归零，说明：

扩容操作会释放之前积攒的全部能量用于支付复制成本，摊销代价是常数。

这是你要的结果。

✚ 普通插入时：

- $(n \rightarrow n+1)$
- (m) 不变

$$\Phi(D_i) - \Phi(D_{i-1}) = 2$$

也就是说普通插入每次都把势能提高一个常数金额，这些钱就被“存入账户”，以后用于支付扩容。

所以这个势能函数自动满足：

普通插入累积能量 → 扩容释放能量 → 平滑成本。

为什么要选这个？因为它刚刚好做到两件事：

1. 普通插入积累的势能大小可以抵消一次扩容时复制的元素数（线性代价）
2. 扩容后势能会重置为接近零，不会暴涨或为负

3. 如何设计势能函数？（给你最实用的套路）

方法论：势能函数 = “未来可能需要付的代价的估计”

你可以直接照下面流程使用（非常常用且高效）：

✓ 通用设计套路（非常实用）

Step 1：找出现贵得离谱的“昂贵操作”

如表扩张中的 **扩容复制 k 个元素** $\rightarrow$ 实际代价 $O(k)$

Step 2：确定你希望每次普通操作积攒多少能量

例如普通插入希望给 2 元钱放到账户里。

Step 3：定义势能为“已经积攒的能量 = 未来可能要付的钱”

例如：

- 一旦 n 增大，到达扩容临界点，你未来会支出 m 次复制，因此势能应与 n 和 m 有某种关系。
- 所以用 $2n - m$ 形式。

Step 4：验证两种操作的摊销代价：

- 普通操作摊销代价 = 常数
- 昂贵操作摊销代价 = 常数

能验证成功的势能函数就是好势能函数。

势能函数的直觉总结

你要记一个非常重要的思想：

势能方法的目的是“推导”势能函数，而是“构造”一个能保证摊销代价有上界的函数。

因此不是唯一的、不是必须的，只要满足：

- 普通操作积累能量
- 昂贵操作释放能量
- 每次操作摊销代价保持 $O(1)$

就是一个正确的势能函数。

QA3

给出常见摊销分析问题的分析模板（聚集方法+记账方法+势能法），然后给出“势能方法典型考试题的解法套路”

第一部分：常见摊销分析问题的“三大方法通用模板”

（聚集方法 + 记账方法 + 势能方法）

你可以直接照这个模板写任何摊销分析题，例如：

- 动态表扩张（扩容/缩容）
- 栈操作 / 栈反向 / 两个栈实现队列
- 二叉树斜向/伸展树
- 多次 push/pop/min 操作
- 多次合并链表等

我写得非常模板化，你照抄即可得到规范答案。

1. 聚集方法（Aggregate Method）

写法模板

1. 列出操作序列：

假设有 (n) 次操作（如插入、删除等多种类型）。

2. 分析总实际代价 $T(n)$ ：

直接计算所有操作真实代价之和。

常见结构：

- 普通操作：代价 = 常数
- 少数昂贵操作：代价大，但发生次数少（如扩容发生 $(\log n)$ 次）

3. 求摊销代价:

$$\hat{c} = \frac{T(n)}{n}$$

4. 得出摊销复杂度:

若 $\hat{c} = O(1)$, 则摊销时间为 $O(1)$ 。

✚ 示例模板 (可套用在任何题)

昂贵操作发生的次数有限, 因此

$$T(n) = O(n)$$

所以每次操作的摊销代价为

$$\hat{c} = \frac{T(n)}{n} = O(1)$$

■ 2. 记账方法 (Accounting Method)

✚ 写法模板

1. 为每类操作设置摊销费用 $\hat{c}_i$

- 普通操作: 设置为某个常数 (如 3)
- 昂贵操作: 设置为某个稍大的常数 (如 5)

(要求剩余余额永不为负)

2. 解释为什么这个费用是足够的:

每次普通操作的摊销费用中:

- 一部分支付当前操作
- 剩余部分“存入账户”, 用于支付未来昂贵操作的真实成本

3. 证明账户余额不为负:

通常给出一段推导:

例如动态数组:

- 普通插入存 2 元

- 扩容时需要的复制成本可完全由之前存的钱覆盖
因此余额不为负。

4. 得出结论：

$$\hat{c}_i = O(1)$$

✚ 示例答案框架（考试可直接写）

为普通操作收取 3 元摊销费用，其中：

- 1 元立即用于当前真实代价
- 2 元存入账户

当发生昂贵操作（如扩容）时，之前累积的资金足以支付所有复制成本，因此账户余额始终不为负。

故摊销代价为常数 ($O(1)$)。

■ 3. 势能方法 (Potential Method)

✚ 写法模板（最标准的证明格式）

1. 定义势能函数 ($\Phi(D_i)$)

- 必须 ≥ 0
- 体现“系统中未来可能发生的代价”

例如动态表常用：

$$\Phi(D_i) = 2n - m$$

2. 写出摊销代价公式：

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$$

3. 分别分析不同类型的操作：

- 普通操作：

展开上述表达式，计算 ($\hat{c}_i$)，得到一个常数。

- 昂贵操作：

写出势能变化（通常会变为 0 或下降），证明摊销代价仍然为常数。

4. 最终得到摊销复杂度：

$$\hat{c}_i = O(1)$$

✚ 示例答案框架（可套任意题）

定义势能函数 ($\Phi(D_i) = \dots$)。

可知势能始终非负。

对普通操作：

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}) = O(1)$$

对昂贵操作：势能下降用于抵消真实代价，因此

$$\hat{c}_i = O(1)$$

故摊销时间也是 ($O(1)$)。

⚡ 第二部分：势能方法典型考试题的“万能套路”

下面给你的是遇到任何势能分析题都能秒解的策略。

■ 势能方法万能套路（5步即可解决任何题）

Step 1：找“昂贵操作”

找出耗时高的操作：

- 如扩容
- 伸展树的旋转
- 多次合并
- 某个 rare but expensive 的动作

势能方法的核心就是：
为昂贵操作提前存钱。

Step 2: 找导致“昂贵操作”的条件

例如：

- 动态数组：当 $\text{size} = \text{capacity}$ 时扩容
- 伸展树：深度大 $\rightarrow$ 将被旋转
- 多次 merge：链表长度大 $\rightarrow$ 将被合并

这些条件帮助你设计势能函数。

Step 3: 设计势能函数（最关键的套路）

你只需要记住一句话：

势能 = 距离触发昂贵操作的“剩余工作量”。

常见模板：

1. 动态数组：

$$\Phi = \alpha n - m$$

2. 栈、队列多次反向：

$$\Phi = \text{未被处理的元素数量}$$

3. 伸展树：

$$\Phi = \sum_v \log(\text{size}(v))$$

4. 两个栈实现的队列：

$$\Phi = \text{stack2 中的元素数量} \quad (\text{或 stack1 的大小})$$

Step 4: 计算摊销代价

对每类操作：

$$\hat{c}_i = c_i + \Delta\Phi$$

如果每项都是常数 → 证明完毕。

一般套路：

- 普通操作：势能增加常数 → 摊销代价常数
- 昂贵操作：势能大幅下降（变为 0）→ 抵消真实成本 → 摊销代价仍常数

Step 5: 检查 $\Phi \geq 0$

最后一步一定要写：

显然 $\Phi(D_i) \geq 0$ 对所有状态都成立。

这是势能法的必要条件。

■ 势能函数设计决策树（非常建议背下来）

当你遇到新的题，按以下方式几秒钟选出势能函数：

① 有“扩容/缩容”？

→ 势能 = 元素数与容量的差

$$\Phi = \alpha n - m$$

② 有“移动元素/反向操作”？

→ 势能 = 被移动但尚未处理的元素数量

③ 有“树结构 + 旋转”？

→ 势能 = 全树的“树高”或“节点 $\log$ 大小”

④ 有“合并/拆分结构”？

→ 势能 = 结构内“权值总和”

⑤ 实在不知道？

→ 势能 = 系统中“潜在的未来工作量”



小结：如何一眼判断是否常数摊销？

只看两点：

- 普通操作 → 势能上涨少量常数
- 昂贵操作 → 势能下降足够多抵消实际代价

满足这两点 → 常数摊销。